AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Michał Tokarz

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

2. Analiza problemu **4**

3. Projektowanie **6**

4. Implementacja **9**

5. Wnioski **10**

Literatura **11**

Spis rysunków **11**

Spis tabel **12**

Spis listingów **13**

1. Ogólne określenie wymagań

Celem niniejszej pracy jest zaprojektowanie oraz implementacja dwukierunkowej listy powiązanej w języku C++, operującej na stercie. Głównym założeniem projektowym jest praktyczne wykorzystanie paradygmatu programowania obiektowego oraz wzorców projektowych, w szczególności wzorca Fabryki (Factory) do zarządzania tworzeniem obiektów oraz wzorca Iteratora do bezpiecznej nawigacji po elementach kolekcji. Równie ważnym celem edukacyjnym jest zapoznanie się z systemem kontroli wersji Git (w tym obsługą platformy GitHub) oraz narzędziem Doxygen do tworzenia dokumentacji.

W ramach projektu chcemy uzyskać w pełni funkcjonalny program wieloplikowy, w którym każda klasa posiada wyodrębniony plik nagłówkowy i implementacyjny. Skupiamy się na zrealizowaniu podstawowych operacji na liście: dodawaniu elementów na początek, na koniec oraz pod wskazany indeks, a także na wyświetlaniu zawartości listy. System musi poprawnie zarządzać pamięcią (używając operatorów `new` i `delete`), aby zapobiec wyciekom.

Przewidywanym wynikiem pracy jest działająca i stabilna aplikacja konsolowa, poprawnie przechodząca testy w funkcji `main`. Ponadto, efektem prac będzie utworzone repozytorium Git zawierające odpowiednią historię commitów (dokumentującą etapy powstawania kodu) oraz automatycznie wygenerowana dokumentacja techniczna w formacie PDF przy użyciu oprogramowania Doxygen i środowiska LaTeX.

2. Analiza problemu

Analiza algorytmu i narzędzia

Zastosowanie algorytmu

Lista dwukierunkowa jest używana tam, gdzie wymagane jest częste wstawianie i usuwanie elementów w środku kolekcji oraz nawigacja w obu kierunkach. Przykłady to: historia przeglądarek internetowych (przyciski wstecz/dalej), listy odtwarzania multimediów oraz mechanizmy „Cofnij/Ponów" w edytorach.

Sposób działania programu

Struktura opiera się na dynamicznie alokowanych węzłach (sterta). Każdy węzeł posiada wartość oraz wskaźniki na element poprzedni i następny. Proces tworzenia węzłów jest zarządzany przez wzorzec Fabryki, a bezpieczne poruszanie się po liście zapewnia wzorzec Iteratora. Zwalnianiem pamięci zajmuje się destruktor.

Przykład wykorzystania (wstawienie pod indeks 1)

Załóżmy wstawienie wartości `20` pod indeks `1` na liście posiadającej już dwa elementy.

1. **Stan wyjściowy:** `[10] <-> [30]`.
2. **Szukanie pozycji:** Iterator przechodzi na indeks 1, wskazując na węzeł `[30]`.
3. **Tworzenie i linkowanie nowego węzła:** Fabryka tworzy węzeł `[20]`. Jego poprzednikiem staje się węzeł `[10]`, a następcą węzeł `[30]`.
4. **Aktualizacja sąsiadów:** Wskaźnik „następny" węzła `[10]` i wskaźnik „poprzedni" węzła `[30]` są aktualizowane tak, aby wskazywały na `[20]`.
5. **Wynik końcowy:** `[10] <-> [20] <-> [30]`.

Opis wykorzystanych narzędzi

- **Git** – system kontroli wersji śledzący historię zmian w plikach, umożliwiający tworzenie zapisów (commitów) oraz cofanie błędów.

- **GitHub** – platforma hostingowa służąca jako zdalne repozytorium do synchronizacji i archiwizacji projektu z różnych lokalizacji.
- **Doxygen** – program automatycznie konwertujący odpowiednio sformatowane komentarze z kodu C++ na gotową dokumentację PDF (poprzez LaTeX).

3. Projektowanie

Środowisko pracy i architektura

Narzędzia i wykorzystane technologie

Podczas realizacji projektu wykorzystano następujący stos technologiczny:

- **Język programowania:** C++ (w standardzie C++17).
- **Kompilator:** `clang++` (Apple LLVM) dostępny natywnie w systemie macOS.
- **Środowisko programistyczne (IDE):** Visual Studio Code wraz z rozszerzeniami C/C++ oraz wbudowanym terminalem powłoki `zsh`.
- **Biblioteki:** Projekt opiera się wyłącznie na standardowej bibliotece języka C++ (m.in. `<iostream>`). Nie wykorzystano żadnych zewnętrznych bibliotek.
- **Narzędzia dodatkowe:** System kontroli wersji Git, platforma GitHub oraz generator dokumentacji Doxygen wraz z dystrybucją LaTeX. Zgodnie z wymogami projektu, program powstał bez wsparcia narzędzi sztucznej inteligencji.

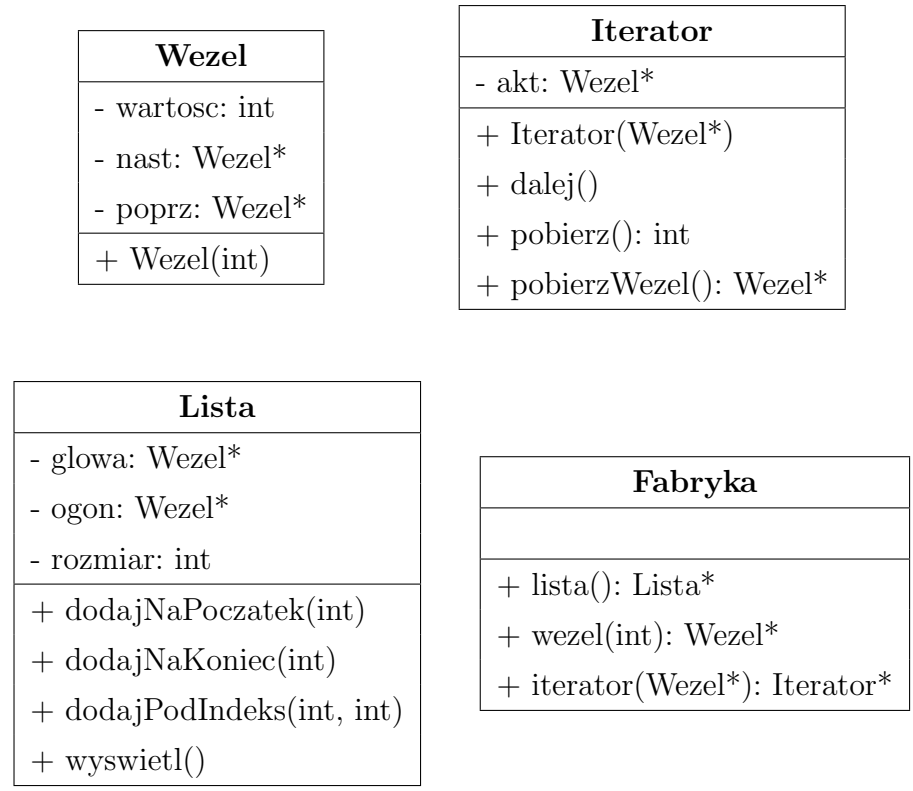
Szczegółowe ustawienia kompilatora

Ze względu na wieloplikową strukturę projektu (wzorzec Factory, Iterator, węzeł oraz lista w osobnych plikach `.cpp` i `.h`), proces budowania został skonfigurowany w pliku `tasks.json` środowiska VS Code. Najważniejsze użyte parametry kompilatora to:

- `-std=c++17` – wymuszenie użycia standardu C++17.
- `${workspaceFolder}/Projekt/*.cpp` – automatyczne łączenie (linkowanie) wszystkich plików implementacyjnych znajdujących się w folderze roboczym, co zapobiega błędom typu *undefined reference*.
- `-g` – dołączenie symboli debugowania, umożliwiających śledzenie alokacji pamięci na stercie i potencjalnych wycieków.

Diagram klas (UML)

Architekturę rozwiązania obrazuje poniższe zestawienie klas (przedstawione w formie blokowej):



Relacje między klasami: Klasa **Fabryka** odpowiada za tworzenie obiektów klas **Lista**, **Wezel** oraz **Iterator**. Klasa **Lista** zarządza obiektami typu **Wezel** (relacja agregacji), natomiast **Iterator** służy do nawigacji i posiada bezpośredni wskaźnik na **Wezel**.

Sposób użycia narzędzi (Git i Doxygen)

Obsługa systemu Git: Narzędzie to wykorzystano z poziomu wbudowanego terminala poprzez interfejs CLI (Command Line Interface). Praca opierała się na następującym przepływie (workflow):

1. **Inicjalizacja i dodawanie:** Komendą `git init` utworzono lokalne repozytorium. Pliki były śledzone poprzez `git add <nazwa_pliku>`.

2. **Zatwierdzanie zmian:** Wykonywano liczne commity (np. `git commit -m "Dodanie klasy Wezel"`), aby zbudować pełną historię pracy, plik po pliku.

3. **Nawigacja w historii:** Aby cofnąć się o zadany etap, używano polecenia `git checkout HEAD~2`. Usuwanie zbędnych commitów i resetowanie stanu realizowano komendą `git reset --hard`.

4. **Praca zdalna:** Po podpięciu repozytorium GitHub (`git remote add origin`),

zmiany wysyłano komendą `git push origin main`, a pobierano z innych lokalizacji za pomocą `git pull`.

Obsługa programu Doxygen: Proces generowania dokumentacji z kodu C++ wymagał następujących kroków:

1. Wygenerowanie pliku konfiguracyjnego poleceniem `doxygen -g Doxyfile`.
2. Dostosowanie ustawień w pliku `Doxyfile` (m.in. `EXTRACT_ALL = YES`, `GENERATE_LATEX = YES`, język polski).
3. Uruchomienie narzędzia komendą `doxygen Doxyfile`, co spowodowało parsowanie komentarzy i wygenerowanie plików LaTeX w folderze `latex`.
4. Skompilowanie plików wyjściowych poleceniem `make` do gotowego dokumentu `refman.pdf`.

4. Implementacja

5. Wnioski

Spis rysunków

Spis tabel

Spis listingów